PACKAGE 11 / VOLUME 10

Start here

Your guide to Package 11

20 lessons · 60 quiz questions · 20 labs

An entry point to the final volume, its tested examples and its thesis-scale review.

September 28, 2026
Graduate self-study specialization

Computer Science and Computer Engineering for Games
Edition: September 28, 2026 · Modules 46–50 · Checkpoint J

This is the final volume of the ten-volume self-study sequence. It expands the existing manifest without changing lesson IDs or replacing earlier volumes. You will connect architecture, timing, verification and trustworthy measurement, then execute and defend one primary research project. A reference implementation is a starting point to inspect and challenge, not your completed thesis.

Open these first

Resource	Purpose
`pdf/Volume_10_Student_Handbook.pdf`	Twenty developed lessons, worked reasoning, reading assignments and a three-question quiz after each lesson.
`pdf/Volume_10_Lab_Workbook.pdf`	Twenty practical labs, five integrated module projects and ten transfer problems.
`pdf/Volume_10_Answer_Key.pdf`	Sixty explained quiz answers, lab review guidance and transfer-problem solutions. Keep closed during first attempts.
`pdf/Gate_J_Thesis_Defense_and_Handoff.pdf`	Thesis scoping, evaluation protocol, artifact requirements, defense and independent review.
`pdf/Volume_10_Math_and_Reference.pdf`	Equations, units, evidence boundaries, glossary and primary-source reading directory.
`browser/index.html`	Offline handbook, searchable lessons and local progress checkboxes. No installation or account required.

The editable text is in `source/`; the structured lesson data is in `curriculum.json`. `templates/` contains learner-fillable research and review records. `code/` contains reference models, eight deliberately unfinished student functions, tests and optional RTL/EDA inputs. `examples/` contains regenerated **synthetic** results, not game benchmarks or physical-design measurements.

Begin with a clean copy

Extract the ZIP to a normal writable directory. Keep one untouched copy and create your own work directory outside the release. Open the offline reader directly, or use the PDF handbook. A browser that disallows local storage may still display lessons; use the Markdown progress record instead. Nothing is uploaded by the reader.

From the extracted package root, on a system with Python available:

```
python code/run_tests.py
python code/demo.py --out my_evidence/reference_outputs
python VALIDATE_PACKAGE.py
```

On systems where the command is named `python3`, substitute it for `python`. The code uses the Python standard library and a Python 3.10-or-newer language baseline; see the validation report for the specific interpreter actually tested. Running code requires no administrator privileges, network downloads or access to hardware devices. Do not run scripts you have not inspected.

The reference suite should pass. The student suite is different:

```
python code/run_tests.py --student
```

It deliberately fails until you implement the eight functions in `code/student/exercises.py`. Read `code/EXERCISES.md`; do not “fix” tests by making them weaker. Tests are a minimum contract, not a proof of every possible implementation.

Prerequisites and first study session

Before 46.1, demonstrate the relevant work from Modules 34, 37–40 and 41–42. The later modules also require the digital-design foundations, timing and verification prerequisites listed in their opening pages. Gate I is the prerequisite for the thesis route. The material is not intended as a beginner’s first day of programming.

For the first session, spend 15 minutes locating your prior evidence and opening a lab record. Spend 35–50 minutes on Lesson 46.1, making a state trace before running the code. Spend another 30–45 minutes reproducing the atomic MSI trace and annotating the C++ publication argument. Finish with its three quiz questions from memory. A correct output alone does not complete the transient-race analysis in the lab.

Repair a prerequisite rather than trying to memorize a missing derivation. Use a fresh transfer problem to show the repair worked. Ask a technical reviewer to inspect concurrency, timing and proof assumptions before building the thesis on them.

What counts as completion

There are **20 numbered lessons, 20 practical labs, 60 numbered lesson quiz questions, five module projects and one final checkpoint**. The ten transfer problems and defense prompts are additional practice, not replacements or extra numbered lessons. Projects reuse lab evidence, and Gate J integrates the thesis work in Modules 49–50; these are not six separate capstones.

Use a first-attempt quiz score to diagnose gaps. The proposed local mastery rule is at least two of three correct with the remaining answer repaired and explained. A lab needs at least eight of ten rubric points **and every mandatory safety/correctness/evidence condition**. These are package review rules, not university grades. A self-reported pass is not an independent review.

For Gate J, execute one thesis, retain a defensible baseline, at least one ablation and one sensitivity study, obtain independent reproduction of the central result, defend the argument and close review actions. A well-executed negative result can pass; an invented positive result cannot. An external tool not run must remain “not run.”

Time planning, without double counting

The original manifest estimates **810–1,380 focused hours** for this volume. The **400–800-hour thesis range is included**, not added. One possible thesis allocation is 40–80 hours for scope, 50–100 for baseline and feasibility, 120–230 for implementation, 90–190 for evaluation, 70–140 for writing and 30–60 for defense and handoff. Those six ranges total 400–800. Adjust scope through review; hours are not a requirement to consume.

A 30–60-page thesis-style report plus appendices is a planning target, not a pass criterion. Hold a 30–45-minute defense with a qualified reviewer or peer panel. The report supplied here is teaching material; it is not a prewritten thesis for submission.

Evidence tiers and limits

The package supplies analytical calculations, finite digital models and synthetic fixtures. Its Python and C++ execution results are recorded in `validation/`. The optional RTL, SBY and ORFS files must be run in an appropriate pinned toolchain before they support a learner claim. Digital CDC tests do not establish MTBF. The serial cache model does not predict real frame times. A completed educational flow is not fabrication-ready commercial signoff.

No electronics purchase, fabrication order or platform modification is required to read the course or run its Python models. Advanced physical-flow labs need a suitable authorized software environment. Do not disable thermal protections, access hardware you do not control, or use restricted libraries without permission. None of this work confers an accredited bachelor's or master's degree.

Suggested working rhythm

Predict, derive, implement, test, inspect a failure, explain, and then review. Keep failed cases and changed hypotheses. Give the answer key a separate review session rather than reading it beside the quiz. Record progress in `templates/PROGRESS.md`, and record evidence paths rather than just checking a box.